INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

SOFTWARE DESIGN AND DEVELOPMENT

COURSE: CLOUD SOLUTIONS DEVELOPMENT

PROJECT:

LeGrandHotel – Web-based hotel management system

DELIVERY No. 1 (25%)

AREQUIPA, 2026

Lecturer:

VÍCTOR CALVO

Presented by:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Peru

2026

Table of contents

1. Scope of the 25% delivery 3
2. Project description 3
   2.1 Technologies 3
3. Architecture and cloud environment 4
   3.1 Environment resources (Terraform) 4
   3.2 Estimated cost 5
4. Defined endpoints 5
   4.1 Single response format 7
   4.2 HTTP status codes 7
5. Separate registration and login 8
   5.1 Automated tests for registration and login 12
   5.2 Guests: Google sign-up and online booking 13
6. Incident log 15
7. Delivery conclusions 16
8. EXTRAS (beyond what is graded at 25%) 16
   8.1 EXTRA: Database diagram (UML) 16
   8.2 EXTRA: Performance and stress tests with k6 (several runs) 18
   8.3 Analysis and bottleneck found 21
   8.4 EXTRA: Code encryption (obfuscation) 21
   8.5 EXTRA: Developer view 22
   8.6 EXTRA: Error and status screens 22
   8.7 EXTRA: Security and quality ahead of schedule 23
   8.8 EXTRA: Mobile app design (Android and iOS) 24
9. Next steps (80% delivery) 28

1. Scope of the 25% delivery

This delivery presents only the items graded for week 9, according to the document «Steps for presenting deliverables, weeks 9-11-13». Everything else already completed is presented at the end, in section 8, marked as EXTRA, as the lecturer requested.

Graded item (25%)	How it is met	Evidence in this report	Status
Cloud environment ready (AWS account, VPC, EC2, RDS)	Infrastructure as code with Terraform: VPC with public and private subnets, EC2, RDS MySQL 8, Security Groups and SSM	Section 3: diagram, resources and estimated cost	Defined as code (Terraform)
Endpoints defined	Routes in api/routes/api.php under /api/v1, single JSON format and OpenAPI specification	Section 4	Done
Separate, working registration and login	Staff: POST /auth/register (Administrator and Developer) and POST /auth/login. Guests: sign-up and sign-in with Google plus their own details (ID number and phone). Each with its own validation and screen: /registro, /login and /completar-registro	Section 5 (flows, screenshots and automated tests)	Done

Repository: https://github.com/Josephml23/LeGrandHotel

2. Project description

LeGrandHotel is a web-based hotel management system. It covers rooms and room types, bookings and stays (check-in and check-out), parking, guests, staff and inventory. It has two kinds of account. Hotel staff (Administrator, Receptionist, Storekeeper and Developer) sign in with email and password, and their accounts are created by the Administrator or the Developer. Guests (Client role) create their account with Google, add their ID number and phone, and book online: they see each room type with what it includes, how many rooms are free and the total. Besides the website, a mobile app for Android and iOS was designed that will use the same API (section 8.8).

2.1 Technologies

Layer	Technology
Backend (API)	Laravel 12 · PHP 8.3 (Docker) · Sanctum (tokens) · Socialite (Google) · Pest (tests)
Frontend	React 18 · TypeScript · Vite · React Router · TanStack Query · Axios · Vitest
Database and cache	MySQL 8 on Amazon RDS · Redis (ElastiCache, 80% delivery)
Containers	Docker + docker-compose (Nginx, PHP-FPM, MySQL, Redis locally)
Cloud	AWS: VPC, EC2, RDS, S3, CloudFront, ALB, CloudWatch, SSM, ECR
Load testing	k6 with HTML report and charts

Layer	Technology
Repository	Monorepo: api/, web/, infra/, tests/k6/, docs/

3. Architecture and cloud environment

Users only reach CloudFront (website) and the load balancer (API). The API runs on EC2 and the database cannot be reached from the internet: it lives in private subnets and only accepts connections from the API instances.

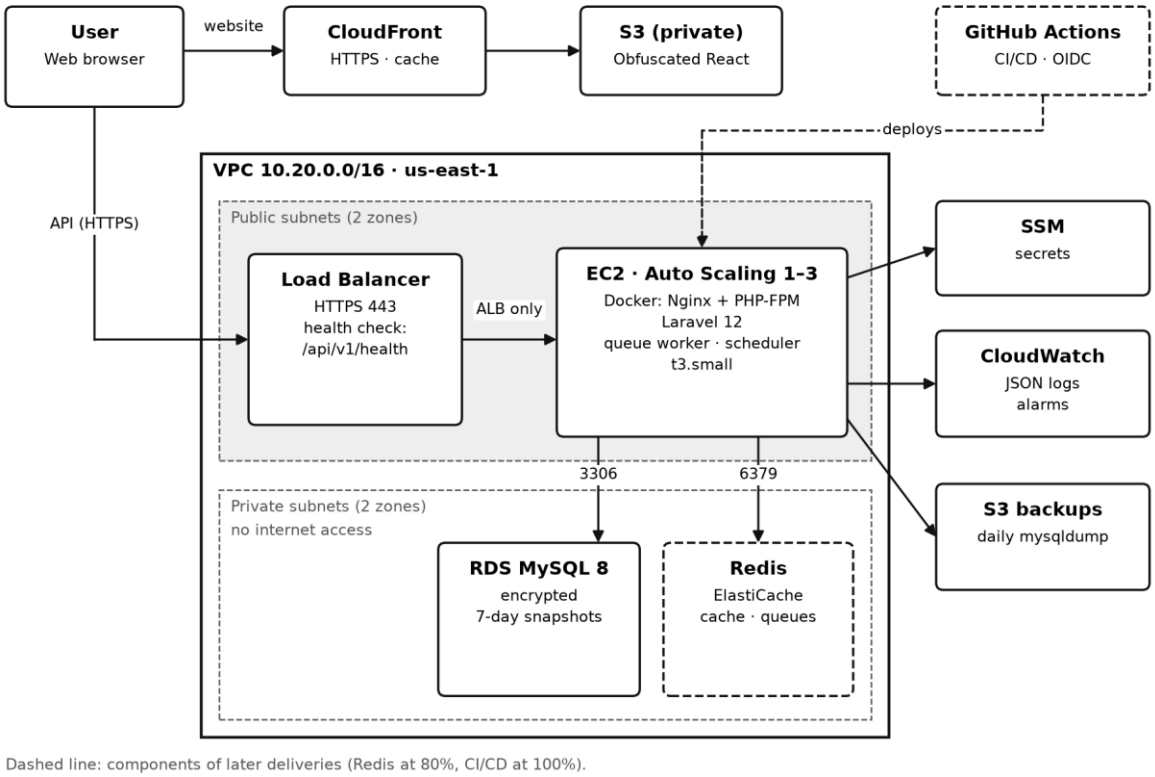


Figure 1. LeGrandHotel architecture on AWS (us-east-1).

3.1 Environment resources (Terraform)

The whole environment is described as code in infra/terraform, so it can be created, reviewed and destroyed repeatably and without manual steps.

Resource	Configuration	File
VPC	10.20.0.0/16, 2 public and 2 private subnets in 2 availability zones	network.tf
EC2	t3.small with Docker (Nginx + PHP-FPM), Auto Scaling from 1 to 3 instances	compute.tf
RDS	MySQL 8, db.t3.micro, 20 GB encrypted gp3, private subnet, 7-day automatic snapshots	data.tf
Security Groups	ALB: 443/80 · EC2: only from the ALB · RDS: 3306 only from EC2	network.tf
SSM Parameter Store	Production secrets and variables (DB password as SecureString)	data.tf

Resource	Configuration	File
Budget	USD 10 alert from day one	monitoring.tf

Snippet 1. *infra/terraform/data.tf and network.tf (summary)*

```
resource "aws_db_instance" "main" {
  engine                  = "mysql"
  engine_version          = "8.0"
  instance_class          = var.db_instance_class   # db.t3.micro
  storage_encrypted       = true
  publicly_accessible     = false                   # no public IP
  db_subnet_group_name    = aws_db_subnet_group.main.name   # private subnets
  vpc_security_group_ids  = [aws_security_group.db.id]
  backup_retention_period = 7                       # daily snapshots, 7 days
}

resource "aws_security_group" "db" {
  ingress {
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]    # only from the API EC2s
  }
}
```

Why it was used: the database is the most sensitive part of the system. With publicly_accessible = false and private subnets it has no internet address; the Security Group only lets traffic into port 3306 from the API instances. The disk is also encrypted and there are 7 days of automatic backups. Because it is written in Terraform, the same environment can be created or destroyed with one command and reviewed before it is applied.

3.2 Estimated cost

Resource	USD/month (24/7)
EC2 t3.small	≈ 15
RDS db.t3.micro + 20 GB	≈ 15
Public IPv4 addresses	≈ 4–8
Total 25% (no ALB or Redis)	≈ 35; ≈ 12 stopping EC2 and RDS outside working hours

4. Defined endpoints

All routes live under /api/v1 and respond with JSON. Except for authentication and health, they require a Sanctum token. Listings accept ?page= and ?per_page= (maximum 50). The full specification is in docs/endpoints/openapi.yaml and can be opened at editor.swagger.io.

Module	Method and route	Access
Health	GET /health	Public
Authentication	POST /auth/register (staff registration)	Admin and Developer (3/min)
Authentication	POST /auth/login	Public (5/min)

Module	Method and route	Access
Authentication	GET /auth/me · POST /auth/logout · POST /auth/refresh	Authenticated
Authentication	GET /auth/google/redirect · GET /auth/google/callback	Public (guests only)
Guest portal	PUT /portal/profile · GET /portal/room-types · GET, POST /portal/reservations · POST /portal/reservations/{id}/cancel	Client
Room types	GET, POST /room-types · GET, PUT, DELETE /room-types/{id}	Admin (write), Front desk (read)
Rooms	GET, POST /rooms · GET, PUT, DELETE /rooms/{id} · PATCH /rooms/{id}/status	Admin, Front desk
Rooms	GET /rooms/available?from=&to=	Admin, Front desk
Guests (guest records)	CRUD /clients	Admin, Front desk
Bookings and stays	CRUD /reservations · POST /reservations/{id}/check-in · POST /stays/{id}/check-out	Admin, Front desk
Parking	CRUD /parking-spots · CRUD /parking-assignments	Admin, Front desk
Inventory	CRUD /inventory-items · POST /inventory-items/{id}/movements	Admin, Storekeeper
Staff and users	CRUD /staff · CRUD /users · PUT /users/{id}/roles	Admin
Reports	GET /reports/occupancy · GET /reports/income	Admin
Developer	GET /dev/overview · /dev/routes · /dev/requests · /dev/errors	Developer, Admin

Snippet 2. *api/routes/api.php (extract)*

```
Route::get('health', HealthController::class);

// Public: staff login and guest sign-in with Google.
Route::post('auth/login', [AuthController::class, 'login'])
    ->middleware('throttle:login');

Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
    Route::get('auth/me', [AuthController::class, 'me']);
    // Staff registration: Administrator and Developer.
    Route::post('auth/register', [AuthController::class, 'register'])
        ->middleware(['role:admin,developer', 'throttle:register']);
    Route::middleware('role:admin')->group(function () {
        Route::post('room-types', [RoomTypeController::class, 'store']);
        // ...
    });
    Route::middleware('role:client')->prefix('portal')->group(/* … */); // guest portal
});
```

Why it was used: all routes are declared in one file, grouped by who can use them. The public routes are staff login, with its own attempt limit, and the Google sign-in. Staff registration sits inside the protected group and requires the Administrator or Developer role; guests can only create their own client account with Google and use the /portal group. Everything else requires a Sanctum token and, where needed, a role. Laravel adds the /api/v1 prefix, so a future API version can live alongside this one without breaking the frontend.

4.1 Single response format

```
Success: { "success": true, "data": {...}, "meta": { "page": 1, "per_page": 15,
"total": 120 } }
Error:   { "success": false, "error": { "code": "ROOM_ALREADY_BOOKED", "message":
"...", "details": {...} } }
```

Snippet 3. *api/app/Support/ApiResponse.php (extract)*

```
public static function success(mixed $data = null, int $status = 200,
                               array $meta = []): JsonResponse
{
    $body = ['success' => true, 'data' => $data];
    if ($meta !== []) { $body['meta'] = $meta; }
    return response()->json($body, $status);
}

public static function error(string $code, string $message, int $status,
                             mixed $details = null): JsonResponse
{
    $error = ['code' => $code, 'message' => $message];
    if ($details !== null) { $error['details'] = $details; }
    return response()->json(['success' => false, 'error' => $error], $status);
}
```

Why it was used: if each controller built its response by hand, the frontend would have to guess the format of every endpoint. With a single class every response has the same shape, and the frontend handles errors in one place by reading error.code.

4.2 HTTP status codes

Code	When it is returned
200 / 201 / 204	Successful read, creation and deletion
400	Malformed JSON body
401	No token, expired token or wrong credentials
403	The role is not allowed (even if the route is typed by hand)
404	Resource or route does not exist (e.g. ROOM_NOT_FOUND)
409	Conflict: room already booked for those dates
422	Validation failed (details per field)
429	Too many requests, with a Retry-After header
500 / 503	Internal error or service down: clear message and incident code, no stack traces

Snippet 4. *api/app/Exceptions/ApiExceptionRenderer.php (extract)*

```
return match (true) {
    $e instanceof ValidationException       => ApiResponse::error(..., 422),
    $e instanceof AuthenticationException   => ApiResponse::error(..., 401),
    $e instanceof AuthorizationException    => ApiResponse::error(..., 403),
    $e instanceof ModelNotFoundException    => $this->modelNotFound($e),      // 404
    $e instanceof ThrottleRequestsException => ApiResponse::error(..., 429),
    $this->isDatabaseDown($e)               => $this->serviceUnavailable($e), // 503
    default                                 => $this->internalError($e),      // 500
};
```

Why it was used: it is the central point that maps any error to an HTTP code and to the single format. This way no error comes out as an HTML page or shows server stack traces; 500

responses include an incident code that is also written to the log, so the exact error can be found without showing internal details to the user.

5. Separate registration and login

There are two kinds of account and each one has its own way in. Staff do not sign themselves up: their account is created by the Administrator or the Developer, and they sign in with email and password. Guests (Client role) create their account with Google and, the first time, add their ID number and phone. Registration and login are separate endpoints, validations (FormRequest) and screens:

	Staff registration	**Staff login**	**Guest with Google**
Endpoint	POST /auth/register	POST /auth/login	GET /auth/google/redirect and /callback; PUT /portal/profile
Who uses it	Administrator and Developer (signed in)	Hotel staff	Guests
Screen	/registro, inside the panel	/login	/login (Google button) and /completar-registro
Data	Name, email, password (twice) and role	Email and password	Name and email from Google; ID number and phone
Validation	Unique email; password ≥ 8 with letters and numbers; the Developer cannot create Developers	bcrypt hash; the account must belong to staff	8-digit ID number; 9-digit phone starting with 9; an ID number cannot be shared between accounts
Result	Creates the account and sends the verification email; does not sign in for the person	60-minute Bearer token and roles	First time: creates the client account; returns the token and the guest can book
Errors	401 not signed in; 403 wrong role; 422 per field; 429 on the 4th registration per minute	401 wrong credentials; 403 client account; 429 on the 6th attempt per minute	Staff cannot use Google; 409 if the ID number belongs to another account; 422 per field

The password is never stored in plain text: the User model uses the «hashed» cast (bcrypt, 12 rounds). After signing in, each role lands on its own home: Administration, Front desk and Storeroom go to the Dashboard, the Developer to their diagnostics view and the Client to «Book». Only guests use Google: if the email belongs to staff it is rejected and the person is asked for their password; guests, in turn, have no password and no access to the internal panel.

Note: the screenshots show the real web interface, which is in Spanish.

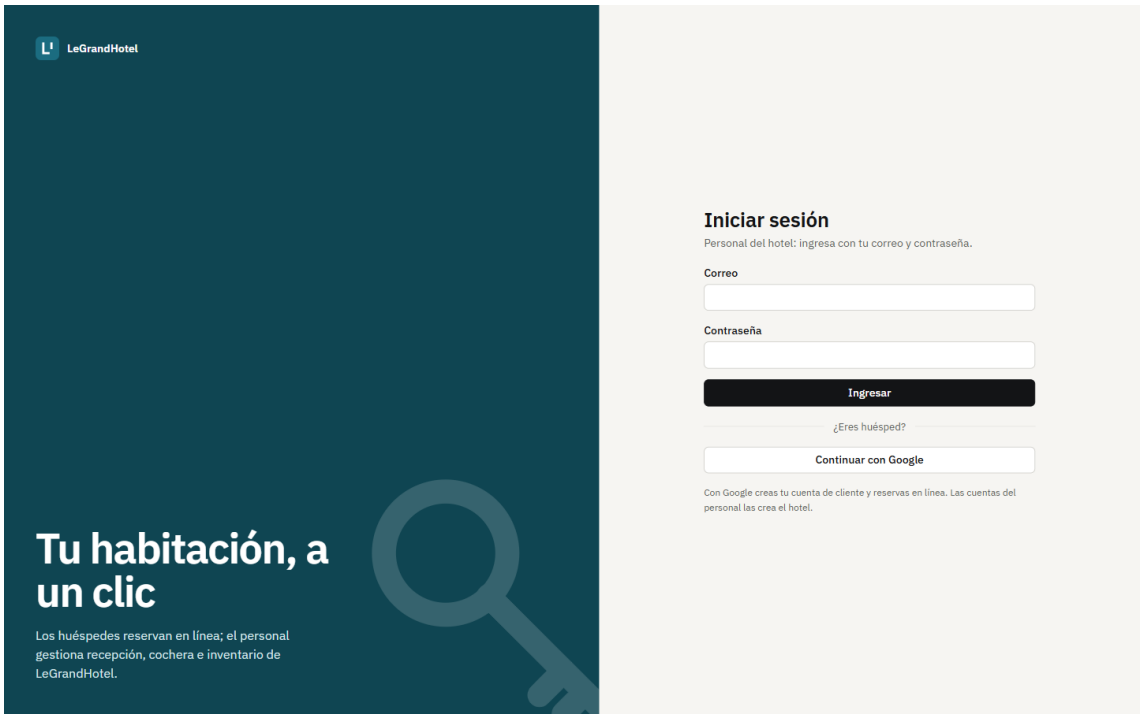


Figure 2. Sign-in (/login): staff with email and password; guests with Google.

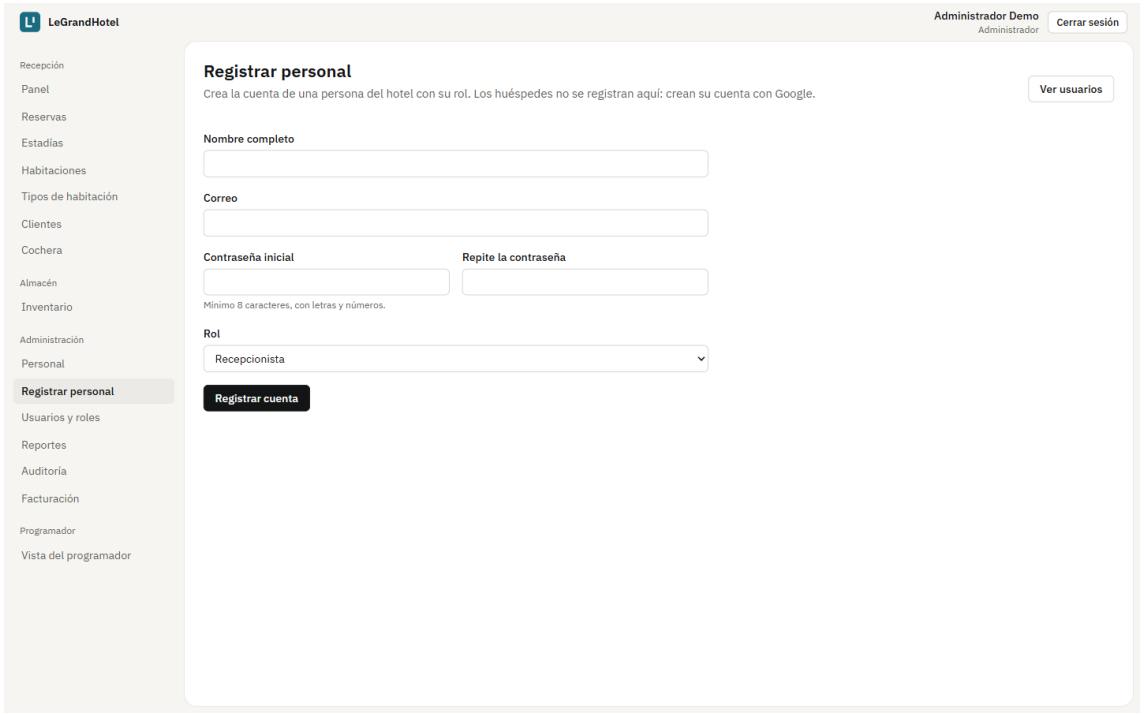


Figure 3. Staff registration (/registro), for the Administrator and the Developer, separate from login.

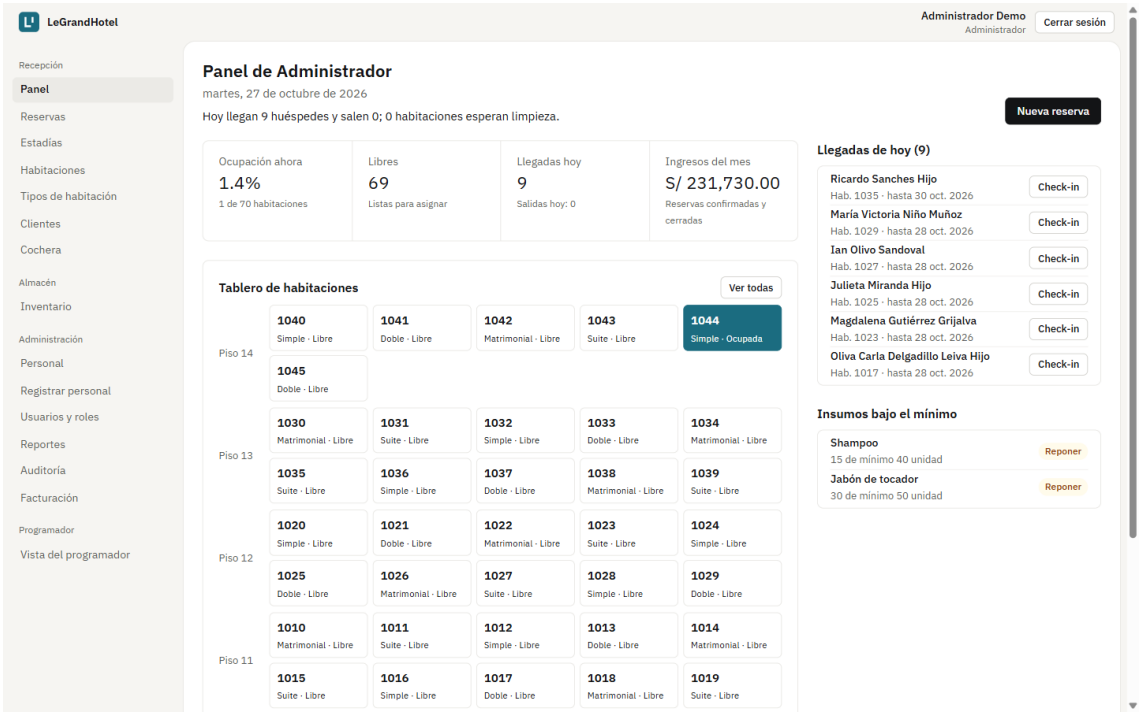


Figure 4. Front desk dashboard after signing in as Administrator.

Snippet 5. *api/app/Http/Requests/Auth/RegisterRequest.php*

```
public function rules(): array
{
    return [
        'name'     => ['required', 'string', 'max:120'],
        'email'    => ['required', 'email', 'unique:users,email'],
        'password' => ['required', 'confirmed',
                       Password::min(8)->letters()->numbers()],
        // Admin: the 4 staff roles; Developer: Front desk, Storeroom and Administration.
        'role'     => ['required', Rule::in(Role::creatableBy($this->user()))],
    ];
}
```

Why it was used: registration has its own validation, different from login. It is validated before reaching the controller: if a field fails, Laravel responds 422 with a message per field and the form shows it next to the field. Role::creatableBy() returns the roles the registering user may assign: the Administrator, all four staff roles; the Developer, all except Developer. It never includes the Client role, which is only created through Google.

Snippet 6. *api/app/Http/Controllers/Api/AuthController.php (extract)*

```
public function register(RegisterRequest $request): JsonResponse
{
    $user = DB::transaction(function () use ($data) {
        $user = User::create([...]);           // password stored with bcrypt
        $user->assignRole($data['role']);       // role chosen by whoever registers
        return $user;
    });
    event(new Registered($user));               // sends the verification email
    return ApiResponse::created(self::userPayloadFor($user));   // no token
}

public function login(LoginRequest $request): JsonResponse
{
    $user = User::where('email', $request->string('email'))->first();
    if (! $user || ! Hash::check($request->password, $user->password)) {
        throw new ApiException('INVALID_CREDENTIALS', '...', 401);
    }
    if (! $user->hasRole(...Role::INTERNAL)) {   // only staff sign in with a password
        throw new ApiException('ACCOUNT_WITHOUT_ACCESS', '...', 403);
    }
    return ApiResponse::success($this->tokenPayload($user, ...));
}
```

Why it was used: they are two separate actions with different responsibilities. Registration is run by the Administrator or the Developer: it creates the account and its role in one transaction (if anything fails, nothing is left half done) and returns no token, because registering someone is not signing in for them. Login compares the password with the bcrypt hash and returns the same message whether the email or the password is wrong, so it does not reveal which emails exist. It also rejects client accounts, which have no password and sign in only with Google. Only login issues a Sanctum Bearer token, which expires after 60 minutes.

Snippet 7. *api/app/Models/User.php and AppServiceProvider.php (extracts)*

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password'          => 'hashed',   // automatic bcrypt
    ];
}

RateLimiter::for('login', fn (Request $r) => Limit::perMinute(5)
    ->by(Str::lower($r->input('email')).'|'.$r->ip()));
RateLimiter::for('register', fn (Request $r) => Limit::perMinute(3)
    ->by($r->user()?->id ?: $r->ip()));
```

Why it was used: with the hashed cast, any password assigned to a user is hashed with bcrypt before it is saved, without relying on each developer remembering to do it. The attempt limits stop brute-force attacks: login allows 5 attempts per minute per email and IP, and registration 3 per minute per administrator; beyond that the API responds 429.

Snippet 8. *web/src/App.tsx (extract)*

```
<Route element={<GuestRoute />}>
  <Route path="/login" element={<LoginPage />} />
</Route>

<Route element={<ProtectedRoute />}>
  <Route element={<ProtectedRoute roles={['admin', 'developer']} />}>
    <Route path="/registro" element={<RegisterPage />} />
  </Route>
  <Route element={<ProtectedRoute roles={['client']} />}>
    <Route path="/reservar" element={<BookPage />} />
    <Route path="/mis-reservas" element={<MyReservationsPage />} />
  </Route>
</Route>
```

Why it was used: in the frontend each way in has its own route. Login is the only public screen. Staff registration sits inside the Administrator and Developer routes, and the guest screens inside the Client routes: without a session they redirect to login, and with another role they show the 403 screen even if the address is typed by hand. The backend enforces the same rules on every endpoint, so hiding the screen is not the only protection.

5.1 Automated tests for registration and login

Test (AuthTest, ClientPortalTest and frontend tests)	Result
The Administrator registers a staff account with a bcrypt password (no token)	Pass
Nobody from outside can register: without a session it responds 401	Pass
The front desk cannot register accounts: responds 403	Pass
Only hotel roles are accepted; invalid data or a repeated email respond 422	Pass
Registration limited to 3 per minute (429 with Retry-After)	Pass
The Developer creates Front desk, Administration and Storeroom accounts, but not Developer accounts (422)	Pass
Google creates the client account the first time; staff cannot sign in with Google	Pass
A client cannot sign in with a password or reach staff routes (403)	Pass
The client adds ID number and phone; an existing front desk record is linked; an ID number of another account returns 409	Pass
Booking by type assigns a room; 409 when none is free or details are missing; guests only cancel their own	Pass
An account without a hotel role cannot sign in (403)	Pass
Login with valid credentials returns a token and roles	Pass
Login with a wrong password responds 401	Pass
Login limited to 5 attempts per minute	Pass
No token or an expired token responds 401	Pass
Frontend (Vitest): staff login and guest Google sign-in are separate; /registro without a session redirects to login, shows 403 for other roles and hides the Developer option from the Developer; the guest adds their details and sees what each type includes	Pass

Overall result: 114 API tests (457 assertions) and 25 frontend tests pass. Static analysis with PHPStan reports no errors and the GitHub Actions pipeline is green.

5.2 Guests: Google sign-up and online booking

Guests sign in with Google from the same /login page. The first time, a Client account is created and they are asked for their ID number and phone; if the front desk already had a record with that ID number, it is linked instead of duplicated. They then pick dates and number of guests and see each room type with what it includes, how many rooms are free and the total for the stay. They book by type and the system assigns a free room; the front desk sees that booking like any other and checks the guest in on arrival. The screenshots show the real interface, which is in Spanish.

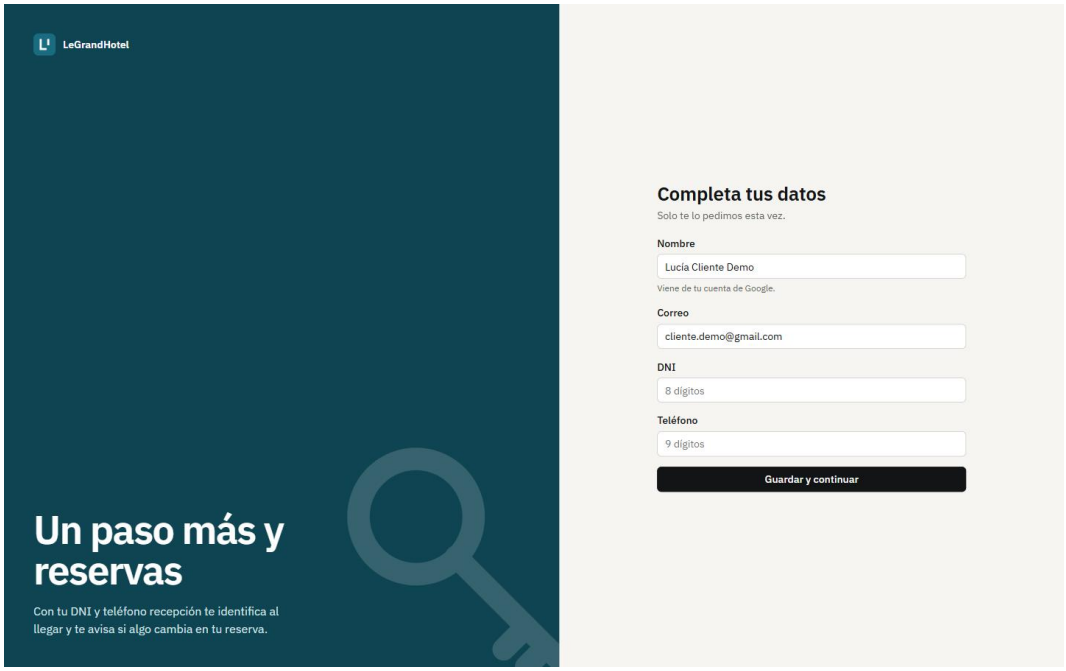


Figure 5. First Google sign-in: the guest adds their ID number and phone (/completar-registro).

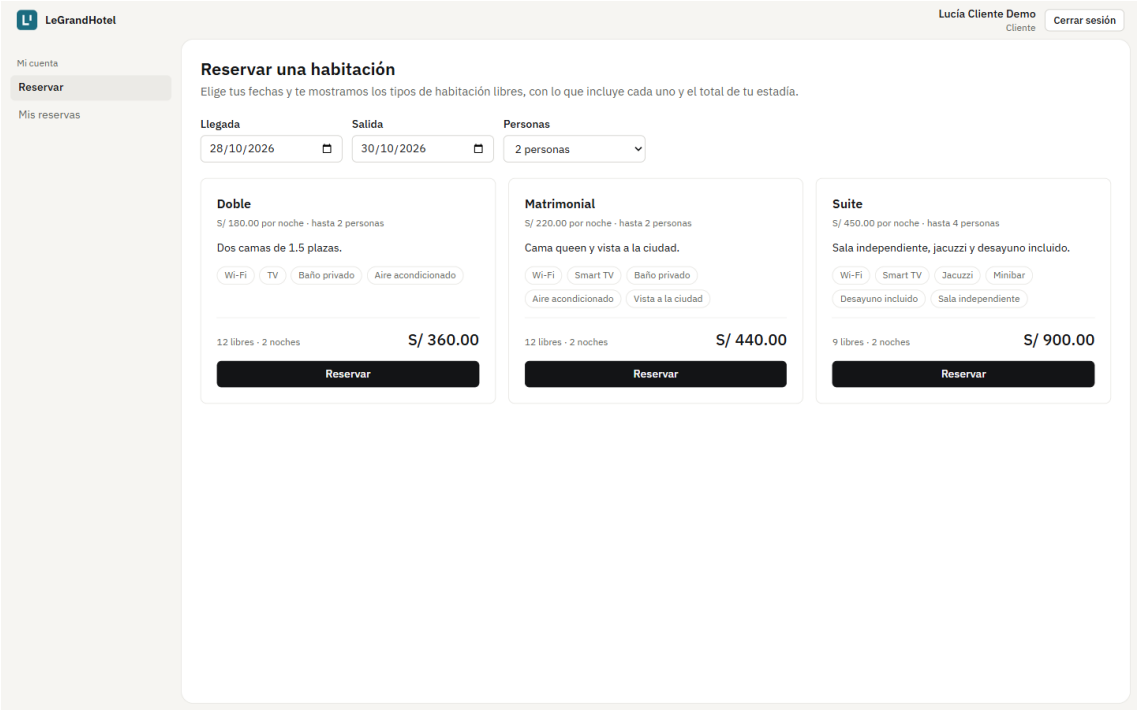


Figure 6. Booking: each room type with what it includes, free rooms and total (/reservar).

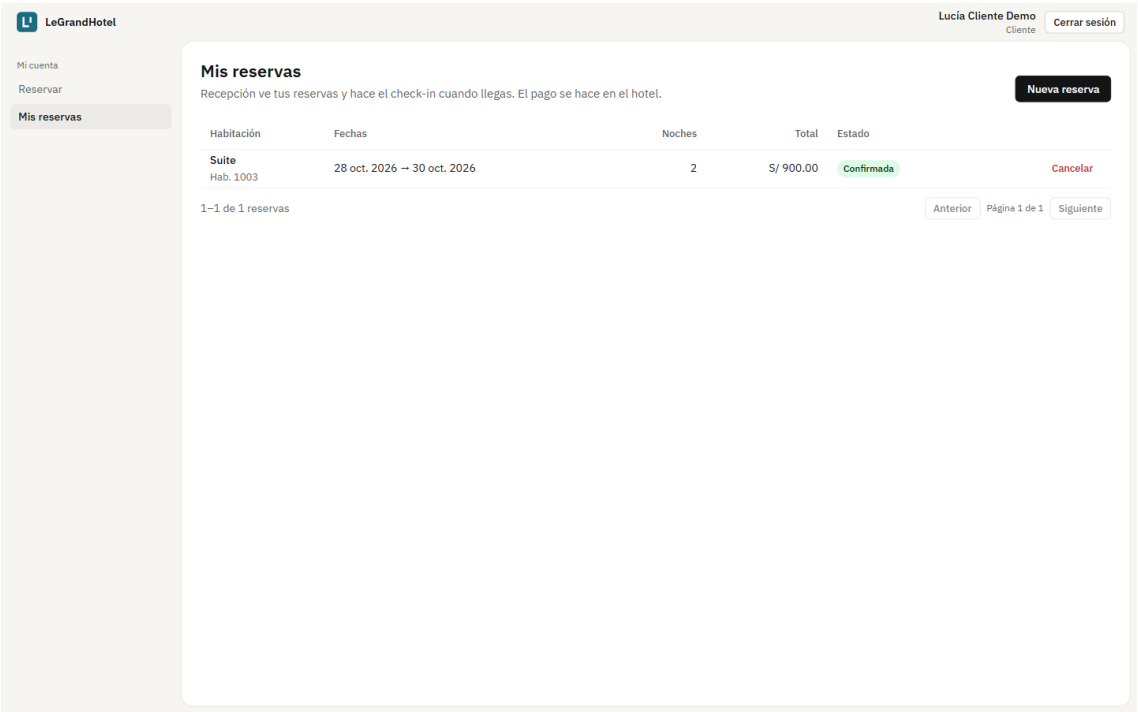


Figure 7. My bookings, which can be cancelled before the arrival day (/mis-reservas).

Snippet 9. *api/app/Http/Controllers/Api/GoogleAuthController.php (extract)*

```
$user = User::where('google_id', $googleUser->getId())
    ->orWhere('email', $googleUser->getEmail())->first();

// Google is for guests only: staff sign in with email and password.
if ($user && ! $user->hasRole(Role::CLIENT)) {
    return redirect()->away("{$frontend}/login?error=google_personal");
}

if (! $user) {                                  // first time: client account
    $user = User::create(['name' => ..., 'email' => ...]);
    $user->assignRole(Role::CLIENT);
}
```

Why it was used: the same button signs guests up and signs them back in. Google has already verified the email, so no extra password is needed. If the email belongs to staff it is rejected, so a hotel account can never get in by any route other than its password, and a guest never receives a staff role.

Snippet 10. *api/app/Http/Controllers/Api/ClientPortalController.php (booking)*

```
$client = $request->user()->client
    ?? throw ApiException::conflict('PROFILE_INCOMPLETE', '...');

$candidates = Room::where('room_type_id', $data['room_type_id'])
    ->availableBetween($data['check_in'], $data['check_out'])
    ->orderBy('number')->pluck('id');

foreach ($candidates as $roomId) {
    try {
        return ApiResponse::created($this->reservations->save([...]));
    } catch (ApiException $e) {           // someone took it at the same moment
        if ($e->errorCode !== 'ROOM_ALREADY_BOOKED') throw $e;
    }
}
throw ApiException::conflict('NO_ROOMS_AVAILABLE', '...');
```

Why it was used: the guest picks the type, not the room number. It reuses the same ReservationService as the front desk, which locks the room row so two people cannot book the same room for the same dates; if a room is taken at that instant it tries the next free one. When none is left it returns a clear 409.

6. Incident log

Every outage, even in development, is logged the same day (docs/incidentes.md).

Date	What failed	Cause	Fix	Time
10/26/2026	Login returned 500 with incident code XG1R7UASWT	«php artisan serve» launched PHP without the pdo_sqlite extension	Local server with php -S loading the extension	≈ 2 min
10/26/2026	The frontend showed the 503 screen	The server pointed to a non-existent index.php	Use api/public/index.php as the router; the 503 screen retried on its own	≈ 1 min
10/27/2026	First k6 run: 100% failed requests	PHP listened only on IPv6 (::1) and k6 resolved localhost to IPv4	BASE_URL=http://[::1]:8000; the runs were repeated	≈ 5 min

7. Delivery conclusions

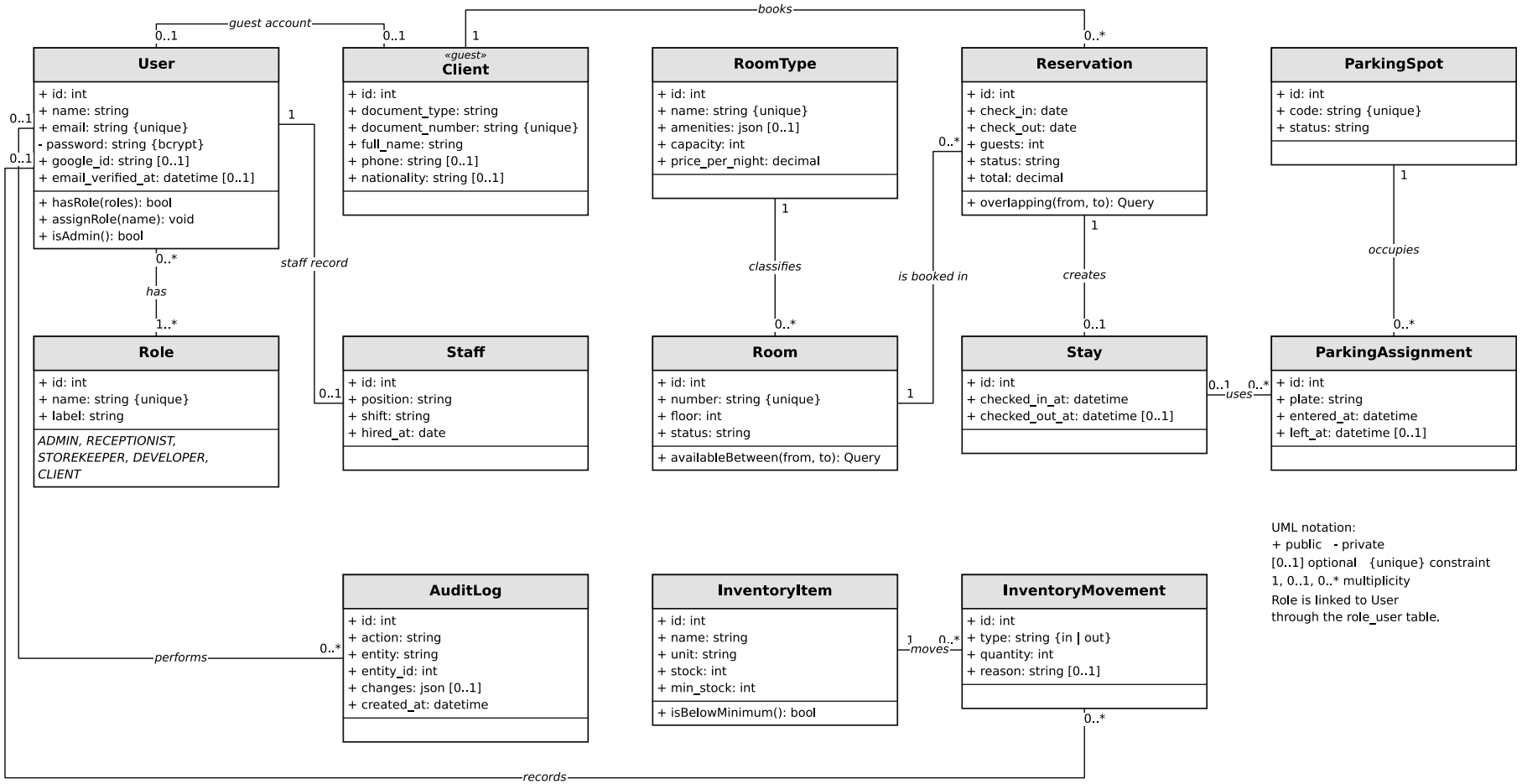
- The three 25% items are implemented: endpoints defined and documented, registration and login separated and tested, and the cloud environment described as code.
- In addition, guests can now create their account with Google and book online, and the Developer can register staff (but not other Developers).
- The local load tests (section 8.2) show that the development server saturates with few users. In the 80% delivery they will be repeated on AWS with Nginx + PHP-FPM, which serves several requests at once.

8. EXTRAS (beyond what is graded at 25%)

Everything in this section is EXTRA: it is not part of what is graded at 25% and is presented as additional progress.

8.1 EXTRA: Database diagram (UML)

UML class diagram of the 13 entities (Laravel models on MySQL 8) with attributes, methods and multiplicities. The role_user join table is the many-to-many association between User and Role; AuditLog and InventoryMovement store which user performed each action; User and Client are linked when a guest creates their account with Google, and RoomType stores the amenities guests see when booking. Indexes required by the plan: rooms(status), reservations(check_in, check_out), reservations(client_id), unique clients(document_number) and inventory_items(name). Editable sources: docs/diagramas/uml-clases.puml (PlantUML) and docs/diagramas/legrandhotel.dbml (dbdiagram.io).



LeGrandHotel class diagram (models in api/app/Models). Editable source: docs/diagramas/uml-clases.puml (PlantUML).

Figure 8. LeGrandHotel UML class diagram.

8.2 EXTRA: Performance and stress tests with k6 (several runs)

Five k6 runs were made against the API on the development machine (Windows, PHP 8.2, PHP built-in server that serves one request at a time, SQLite and file cache). It is not the production server: the runs are used to compare load between runs and to find bottlenecks before moving to AWS.

Scenario (front desk): 70% reads (available rooms, bookings, room types), 20% login and 10% booking creation. Each step ramps up for 10 s, holds the load for 45 s and ramps down for 5 s. Plan target: p95 below 500 ms and 0% errors.

Run	Requests	Errors	p50	p95	Result
Smoke (5 VU, 30 s): health, DB, cache and 401	222	0.00%	203 ms	321 ms	Met (100% of checks)

VU	Requests	Req./s	p50	p95	Max.	Errors	p95 < 500 ms
5	350	4.4	288 ms	528 ms	1,804 ms	0.00%	Not met
10	413	5.2	910 ms	1,309 ms	1,598 ms	0.00%	Not met
25	438	5.2	3,361 ms	3,952 ms	4,197 ms	0.00%	Not met
50	483	5.3	7,016 ms	7,885 ms	8,195 ms	0.00%	Not met

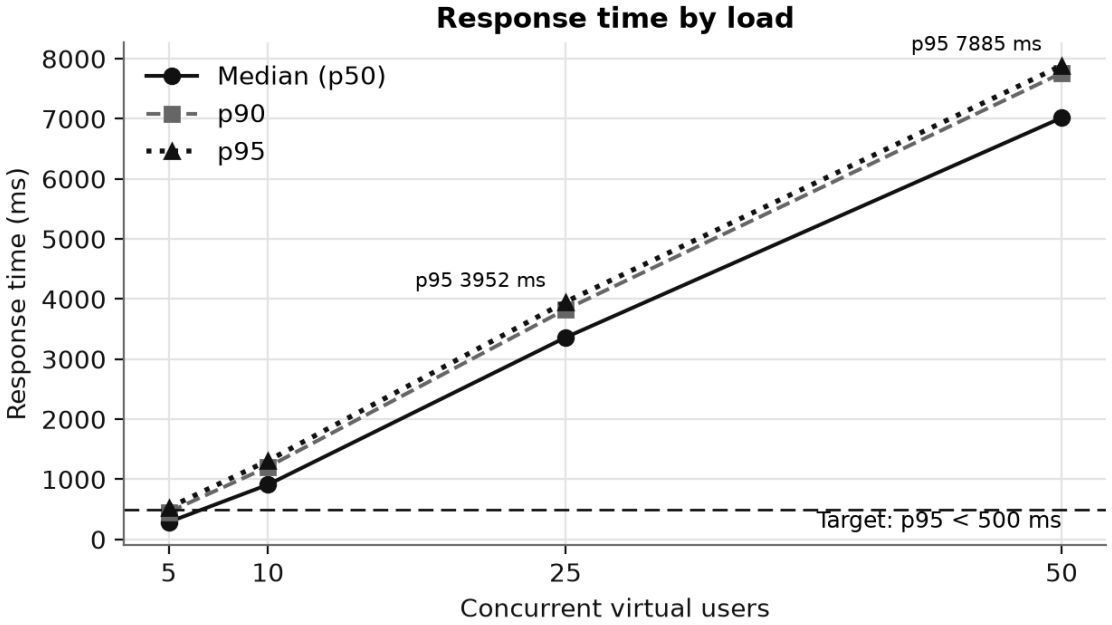


Figure 9. Response time (p50, p90, p95) by number of virtual users.

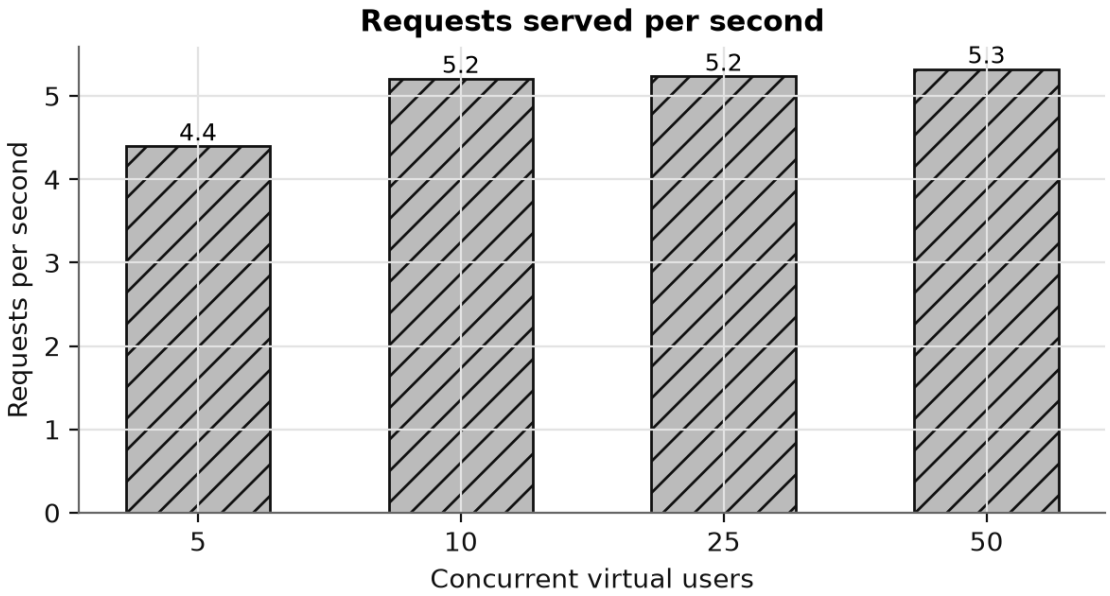


Figure 10. Requests served per second in each run.

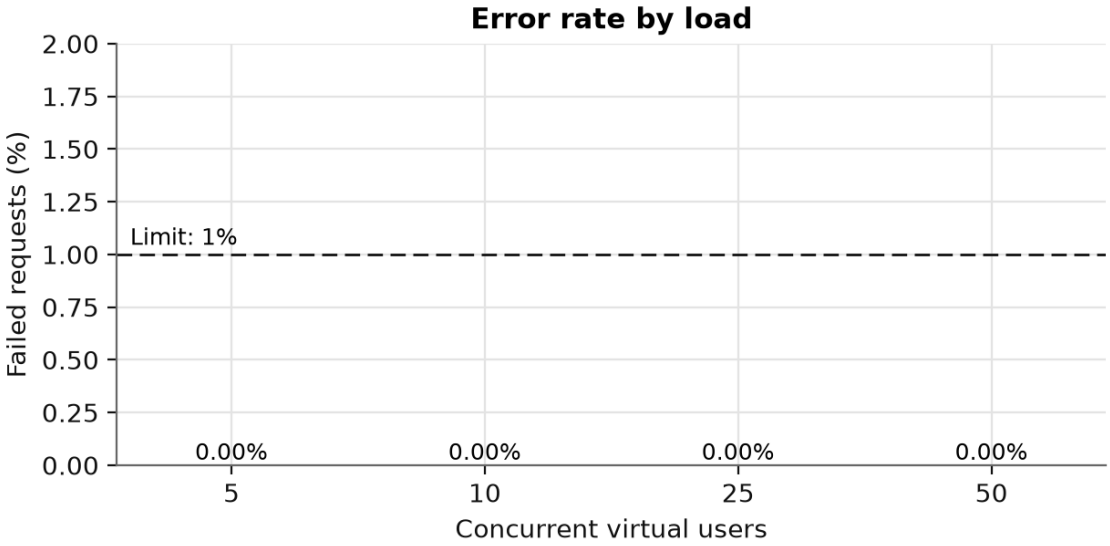


Figure 11. Percentage of failed requests in each run.

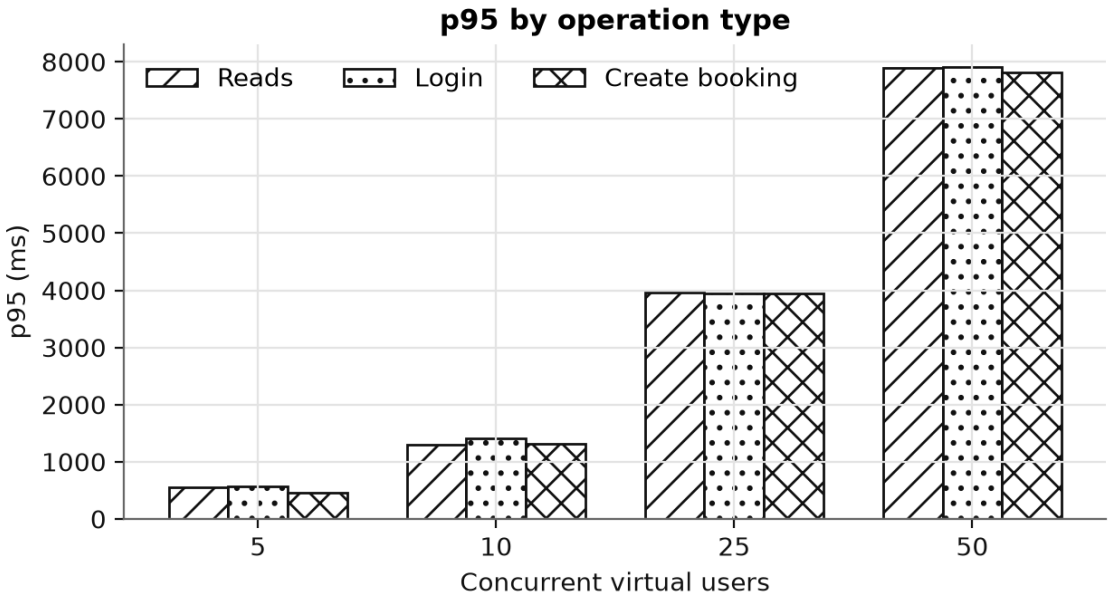


Figure 12. p95 by operation type (reads, login, create booking).

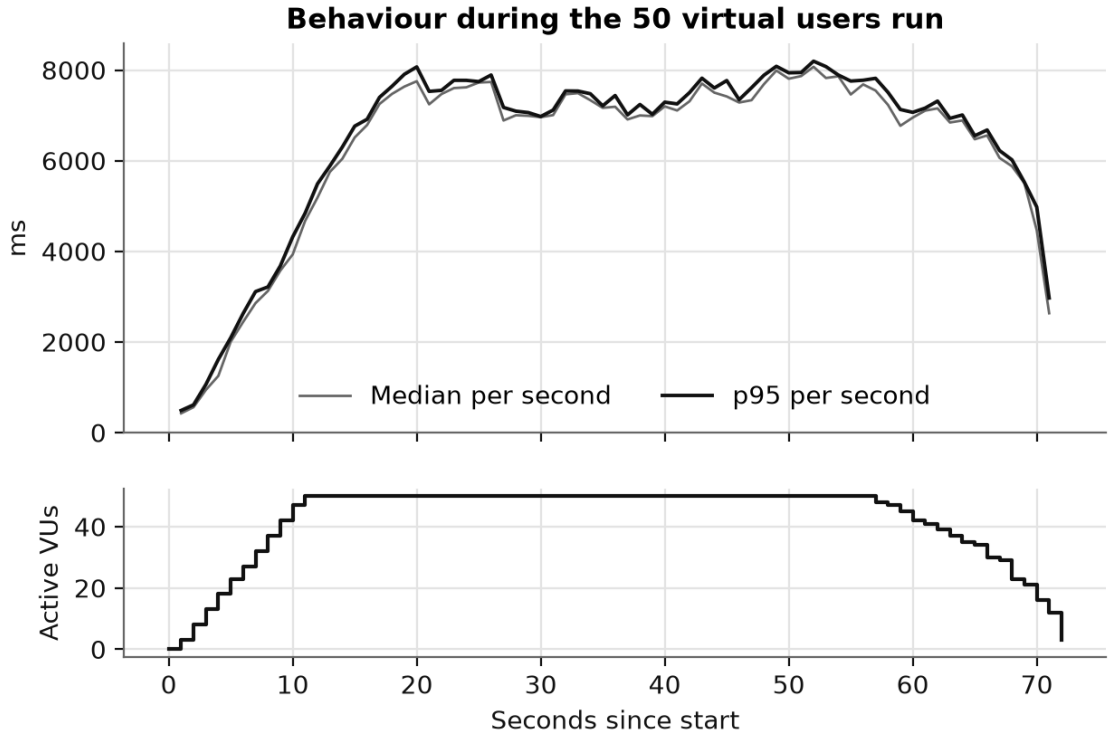


Figure 13. Latency second by second in the 50-user run.

Snippet 11. *tests/k6/reservas.js (extract)*

```
escalon: {
  stages: [
    { duration: '10s', target: VUS },   // ramp up to VUS users
    { duration: '45s', target: VUS },   // hold the load
    { duration: '5s',  target: 0 },     // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],   // target: p95 below 500 ms
    http_req_failed: ['rate<0.01'],     // less than 1% errors
  },
},
```

Why it was used: the same script is run with a different number of users (VUS) to compare runs with each other. The thresholds turn the plan target (p95 below 500 ms and less than 1% errors) into a rule: if it is not met, k6 marks the test as failed without interpreting the numbers by hand.

8.3 Analysis and bottleneck found

- With 5 users the p95 was 528 ms; with 50 users it reached 7,885 ms. Requests per second barely rise (4.4 → 5.3): the server was already at the limit of what it can serve.
- Cause: the PHP built-in server on Windows handles one request at a time. When more users arrive, requests wait in a queue and the response time grows almost linearly, even though each endpoint's code is fast (in the Pest bulk test, the listing of 5,000 bookings responds in under 500 ms).
- With 50 VU, reads, login and booking creation have almost the same p95 (7,890, 7,905 and 7,820 ms). The time is dominated by waiting in the server queue, not by each endpoint's work.
- Despite the saturation there were no errors (0% in every run): the API responds more slowly, but it does not go down or lose requests.
- Fix planned for 80%: Docker with Nginx + PHP-FPM (20 processes), OPcache, MySQL on RDS and Redis for cache and rate limiting. The same runs will be repeated on AWS with 50–100 and then 1,000–5,000 users, to compare before and after.

The interactive HTML reports of each run (generated by k6) are in docs/pruebas/local-*.html and the raw data in docs/pruebas/datos/.

8.4 EXTRA: Code encryption (obfuscation)

The frontend production build is obfuscated with javascript-obfuscator: names are replaced with hexadecimal identifiers and strings are stored base64-encoded. Comparison of the same interface text («No tienes acceso», "You do not have access"):

Build	Command	Times it appears readable
Normal	npm run build:plain	1
Obfuscated (production)	npm run build	0

The backend is not published: it runs on a private server with no access to the code.

Snippet 12. *web/vite.config.ts (extract)*

```
renderChunk(code, chunk) {
  const result = JavaScriptObfuscator.obfuscate(code, {
    identifierNamesGenerator: 'hexadecimal',   // unreadable names
    stringArray: true, stringArrayEncoding: ['base64'],   // encoded strings
    splitStrings: true, compact: true,
  })
  return { code: result.getObfuscatedCode(), map: null }
}
```

Why it was used: a web application's code is downloaded to the browser, so anyone can read it. Obfuscation is applied to the final production bundle to make it harder to copy or study its logic, without changing what it does. Development uses the normal code (npm run build:plain).

8.5 EXTRA: Developer view

The Developer role has a diagnostics screen (/programador) that shows, without logging into the server: the status of the application, the database and the cache; version and configuration (token lifetime, rate limits, CORS); p50/p95/p99 latencies computed from the JSON log; slowest routes; latest requests; errors searchable by incident code, the same one the user sees on the 500 screen; and the list of every API route with its access role. It shows no passwords or keys, and an automated test checks this.

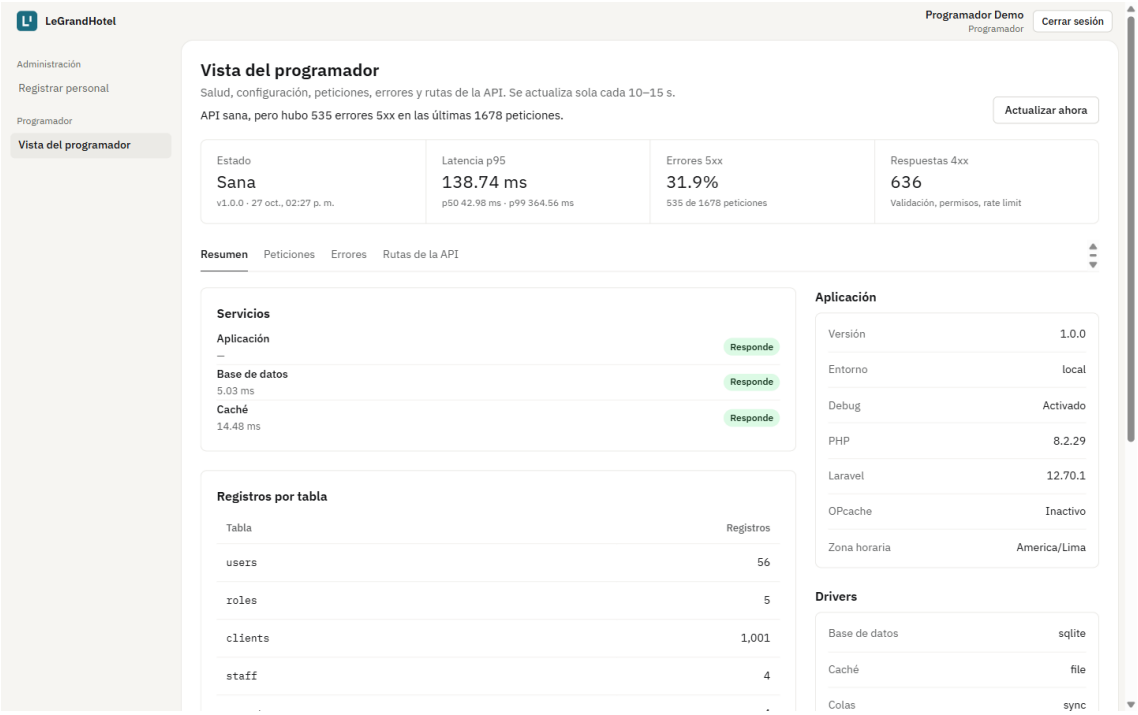


Figure 14. Developer view.

8.6 EXTRA: Error and status screens

Every API error has its own screen, in the system style and with a clear way out: sign in, go back, retry or wait. The 503 screen retries on its own every 30 seconds and the offline screen locks the page without losing what was typed. All 28 screens of the system are in docs/diseno/.

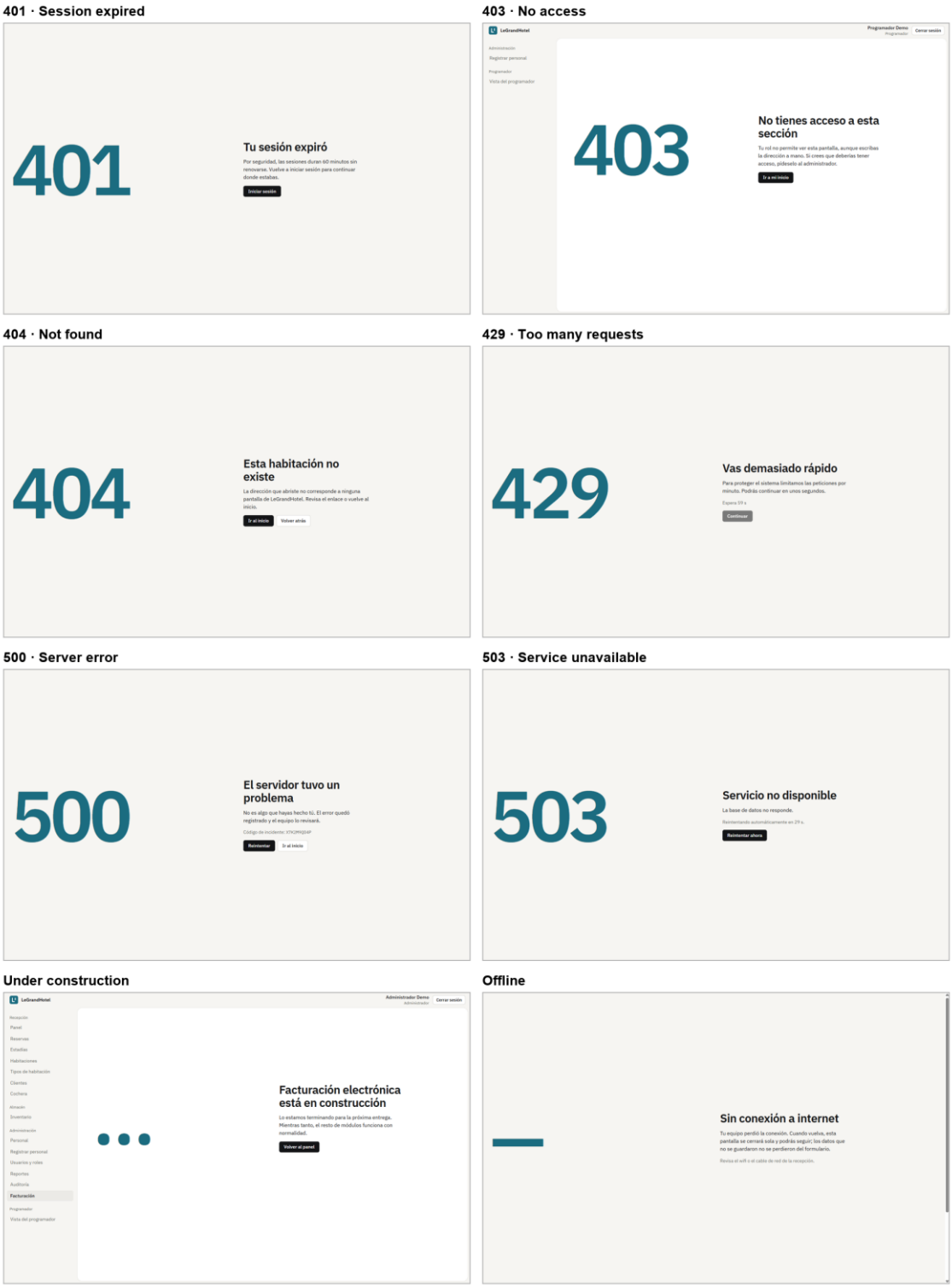


Figure 15. Error and status screens: 401, 403, 404, 429, 500, 503, under construction and offline.

8.7 EXTRA: Security and quality ahead of schedule

- Rate limits: login 5/min, registration 3/min and API 60/min, with a 429 response and Retry-After (an 80% item).
- Guests and staff are kept apart: guests only use /portal and only see and cancel their own bookings; staff cannot sign in with Google and guests have no password.

- Roles and permissions: the role is checked before looking up the record, so a route typed by hand responds 403 without revealing whether the id exists.
- Error screens in React: 404, «Under construction», 403, session expired, 500 with incident code, 503 retrying every 30 s, offline and 429 with a countdown.
- CI/CD on GitHub Actions: style (Pint), analysis (PHPStan), tests (Pest and Vitest), dependency audit and Semgrep on every change. Current audit: 0 vulnerabilities in composer and npm and 0 Semgrep findings.
- Structured JSON logs (one line per request with request_id, route, status code and duration), ready for CloudWatch.

8.8 EXTRA: Mobile app design (Android and iOS)

LeGrandHotel will also have an app for Android and iOS phones. This delivery presents its design: the technology, the architecture, the screens and the build plan. The app code is developed in the 80% and 100% deliveries. It is a tool for hotel staff: it has no sign-up of its own, and guests keep booking from the website with their Google account. It does not replace the web app: it covers the tasks that are better done on the move and uses the same API, without duplicating business logic.

Staff role	What the app will be used for
Receptionist	See the day's arrivals and departures, check guests in and out from the lobby, record the booking of a guest who walks in or calls, change a room's status, record license plates in the parking lot
Storekeeper	Record inventory ins and outs in the storeroom and see supplies below the minimum
Administrator	See the summary dashboard (occupancy, income, alerts). Full management stays on the web

Chosen technology: React Native with Expo and TypeScript. A single codebase produces both apps and reuses what the team already uses on the web: React, TypeScript, TanStack Query, Axios and the same data types.

Option	Pros	Cons	Decision
React Native + Expo	One codebase for Android and iOS; same language and libraries as the web; builds iOS in the cloud without a Mac (EAS Build)	Very specific native features need custom modules	Chosen
Flutter	Good performance and consistent design	Another language (Dart): nothing from the web is reused	Discarded
Native (Kotlin + Swift)	Full access to the system	Two different apps: double the work for 4 people	Discarded
Installable web app (PWA)	No new code	Notifications and secure storage limited on iOS; not in the stores	Complement

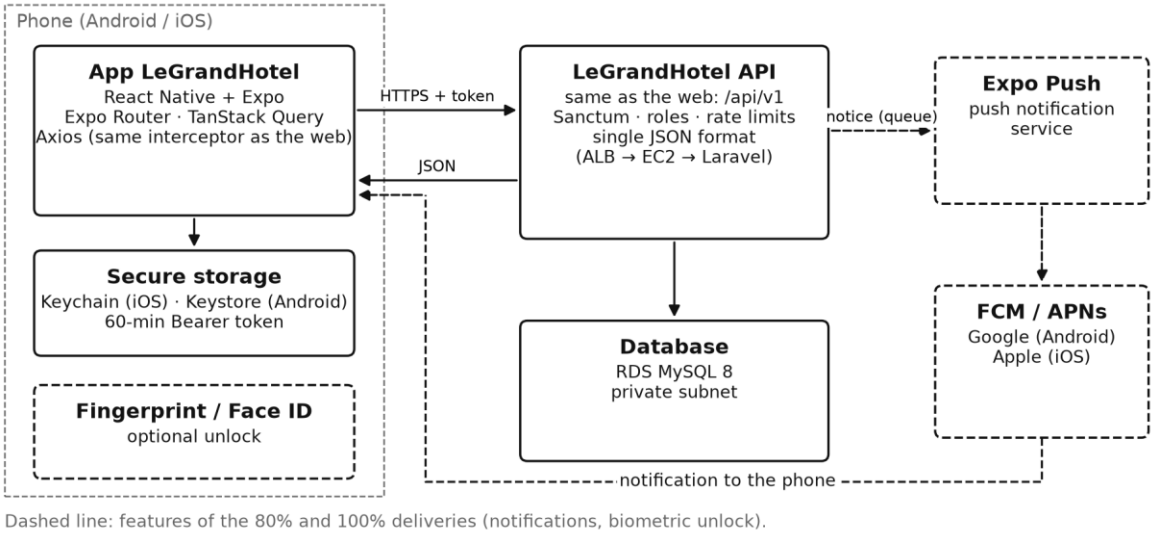


Figure 16. Mobile app architecture: same API and notifications through FCM (Android) and APNs (iOS).

Need	In the app	Web equivalent
Navigation	Expo Router (tab bar per role)	React Router
API calls and cache	TanStack Query + Axios with the same interceptor	Same
Session storage	expo-secure-store: Keychain (iOS) and Keystore (Android)	sessionStorage
Google sign-in (staff accounts only)	expo-auth-session and legrandhotel://auth/callback link	Socialite with redirect
Notifications	expo-notifications → FCM / APNs	—
Unlock	Fingerprint or Face ID (expo-local-authentication)	—
Tests	Jest + Testing Library; Maestro for end-to-end flows	Vitest; Playwright

Changes the API will need (few, because the app uses the same endpoints):

- Register each phone's notification token (POST /devices) and send notices to staff: the day's arrivals, room ready after cleaning and supply below the minimum.
- Google sign-in that returns to the app (deep link) as well as to the web.
- X-App-Version header to ask for an app update if the API changes (426 response).
- Nothing else: login already accepts a device name so each phone has its own token; CORS does not apply to native apps; roles and attempt limits are the same.

Security: the token is kept in the system's secure storage and never in plain text. It is renewed automatically before it expires, with the same POST /auth/refresh as the web. Staff can turn on fingerprint or face unlock. The app only uses HTTPS and contains no secrets: every secret stays on the server.

Delivery	App scope
25% (this one)	Design: technology, architecture, screens per role and sketches

Delivery	App scope
80%	Expo project in mobile/; staff login; front desk («Today» with check-in and check-out, new booking, rooms); notifications; Android test build
100%	Inventory, parking and administrator dashboard; iOS test build on TestFlight; end-to-end flow tests with Maestro; test release on Google Play and the App Store

Publishing: test builds are distributed through Google Play (internal testing) and Apple TestFlight. Publishing requires a Google Play Console account (one-time fee of USD 25) and an Apple Developer Program account (USD 99 per year). The full design is in docs/movil/diseno-app-movil.md.

Mobile app sketches

Main screens in grayscale. They use the same data and rules as the web app; the bottom bar changes with the user's role.

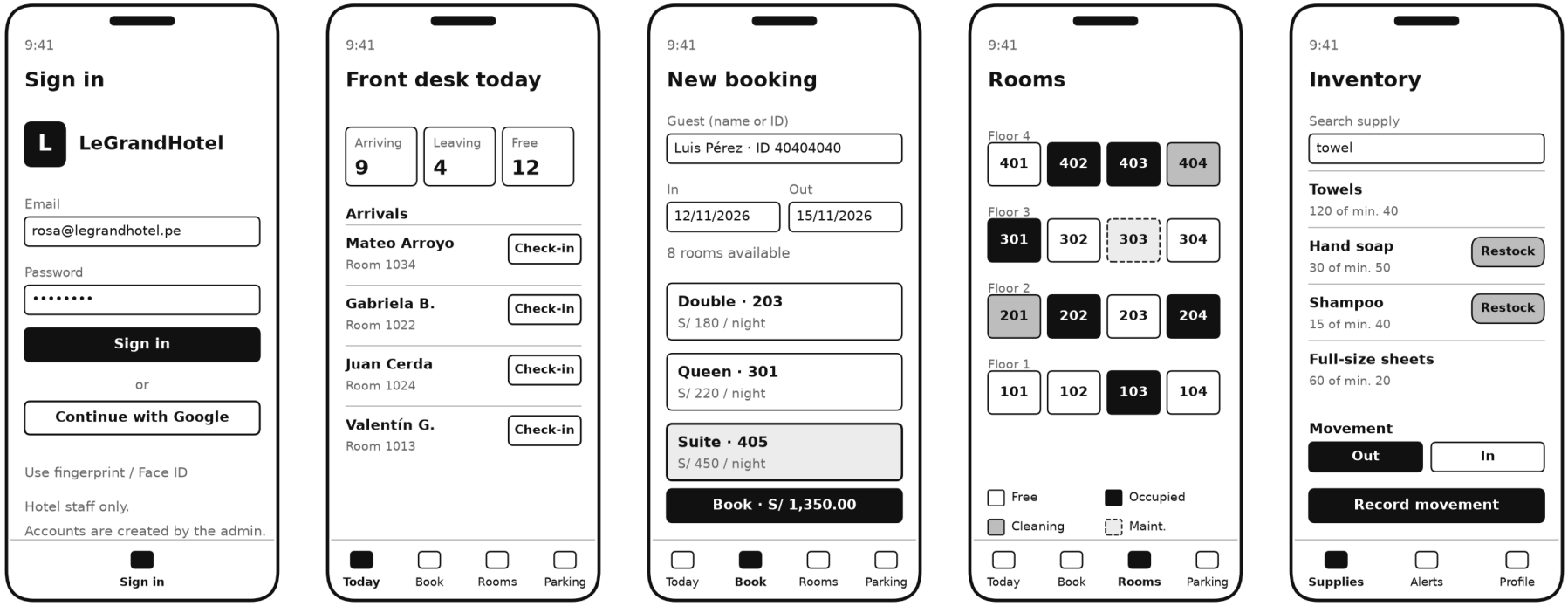


Figure 17. Sketches: staff login; front desk «Today», new booking and rooms; storeroom inventory.

9. Next steps (80% delivery)

- Deploy with the pipeline: Docker image in ECR, frontend on S3 + CloudFront, ALB with HTTPS.
- Repeat the k6 tests on AWS with 50–100 and 1,000–5,000 users, with HTML reports and a before/after comparison.
- Set up Google sign-in (OAuth credentials) and automatic RDS and S3 backups.
- Mobile app: create the Expo project, build the login and front desk screens, enable notifications and produce the first Android test build.